Assignment Solution: Search Algorithms and Local Optimization

Hamza Arif
Reg No: 2024206
AI-202 Assignment 1

Question 1: BFS and DFS Tracing

(a) Breadth-First Search (BFS)

Step	Node Expanded	Queue State	Visited
1	A	[B, C]	A
2	B	[C, D, E]	A, B
3	C	[D, E, F]	A, B, C
4	D	[E, F, G]	A, B, C, D
5	E	[F, G]	A, B, C, D, E
6	F	[G]	A, B, C, D, E, F
7	G	[]	A, B, C, D, E, F, G

From this exploration, the first time we reach G is through the shortest number of steps, which leads to the final BFS path.

Final Path: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ G

(b) Depth-First Search (DFS)

As a result, DFS may reach the goal node faster in terms of exploration steps, but not necessarily through the shortest or most optimal route.

Step	Node Expanded	Stack State	Visited
1	A	[C, B]	A
2	B	[C, E, D]	A, B
3	D	[C, E, G]	A, B, D
4	G	[]	A, B, D, G

Final Path: A → B → D → G

DFS is not optimal because it does not take edge weights or path cost into account; it simply follows depth-first exploration.

Question 2: UCS and A*

(c) Uniform Cost Search (UCS)

Step	Node Expanded	Priority Queue (Node: Cost)
1	A	(C:2), (B:4)
2	C	(B:4), (F:5), (D:10)
3	B	(F:5), (D:9), (E:14)
4	F	(D:9), (G:12), (E:14)
5	D	(E:11), (G:12)
6	E	(G:12)
7	G	—

Optimal Path: A → B → D → E → G

Total Cost: 12

(d) A* Algorithm

Step	Node	g(n)	f(n)
1	A	0	14
2	C	2	13
3	B	4	14
4	F	5	14
5	D	9	15
6	E	11	14
7	G	12	12

Final Path: A → B → D → E → G

The path cost is 12, which matches the true lowest possible cost found in UCS. Despite the flawed heuristic, A* successfully found an optimal path in this specific execution.

(e) Heuristic Analysis

A heuristic is considered admissible if it never overestimates the true cost to reach the goal. However, in this case, we observe that at node E, the heuristic value is greater than the actual remaining cost to the goal, which violates the admissibility condition.

Similarly, for consistency, the heuristic must satisfy the triangle inequality across every edge. When checking the edge from D to E, this condition is also violated, meaning the heuristic is not consistent either.

Conclusion: The heuristic is neither admissible nor consistent.

Question 3: Hill Climbing

(f) Steepest-Ascent Hill Climbing

Hill Climbing is a local search technique that starts from an initial solution and continuously moves towards better neighboring solutions. In steepest-ascent hill climbing, at each step, we evaluate all neighboring states and move to the one with the highest improvement.

Starting from $x = 1$, we observe how the function value improves until no better neighbor exists.

Step	x	f(x)	Neighbors	Move
1	1	15	f(0)=10, f(2)=18	Right
2	2	18	f(1)=15, f(3)=19	Right
3	3	19	f(2)=18, f(4)=18	Stop

Conclusion: The algorithm converges at $x = 3$, which is a local maximum.

(g) Hill Climbing on $g(x)$

In this case, the function has multiple peaks due to its sinusoidal nature. This makes it difficult for hill climbing to consistently reach the global maximum because the algorithm can easily get trapped in a local maximum depending on the starting position.

Simulated Annealing addresses this limitation by introducing controlled randomness. At higher temperatures, the algorithm can accept worse solutions, allowing it to escape local maxima. As the temperature gradually decreases, the algorithm becomes more selective and eventually stabilizes around a good solution.

Question 4: A* Pathfinder Implementation

Python Code

```python
import heapq

grid = [
  [0,0,0,0,1,0,0,0],
  [0,1,1,0,1,0,1,0],
  [0,0,0,0,0,0,1,0],
  [0,1,0,1,1,0,0,0],
  [0,1,0,0,0,1,1,0],
  [0,0,0,1,0,0,0,0],
  [1,1,0,1,0,1,0,1],
  [0,0,0,0,0,0,0,0]
]

start = (0, 0)
goal = (7, 7)

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar():
    pq = []
    heapq.heappush(pq, (0, start))

    came_from = {}
    g = {start: 0}
    nodes_expanded = 0

    while pq:
        _, current = heapq.heappop(pq)
        nodes_expanded += 1

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path, g[goal], nodes_expanded

        for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
            nx, ny = current[0] + dx, current[1] + dy

            if 0 <= nx < 8 and 0 <= ny < 8 and grid[nx][ny] == 0:
                neighbor = (nx, ny)
                new_cost = g[current] + 1

```

```
                if neighbor not in g or new_cost < g[neighbor]:
                    g[neighbor] = new_cost
                    f = new_cost + heuristic(neighbor, goal)
                    heapq.heappush(pq, (f, neighbor))
                    came_from[neighbor] = current

    return None, None, nodes_expanded


path, cost, nodes = astar()

if path:
    print("Path:", path)
    print("Cost:", cost)
    print("Nodes Expanded:", nodes)
else:
    print("No path exists")
    \section*{Graph / Visualization}
```

```
PS C:\Users\hamzz\OneDrive\Documents\GitHub\Semester-4> & c:/Users/hamzz/OneDrive/Documents/GitHub/Semester-4/.ve
nv/Scripts/python.exe c:/Users/hamzz/OneDrive/Documents/GitHub/Semester-4/Ai202/assignment1.py
Path Found!
Cost: 14
Nodes Expanded: 38

Grid Visualization:
S . . # . . .
* # # . # . # .
* * * . . . # .
. # * # # . . .
. # * * * # # .
. . . # * * * .
# # . # . # * #
. . . . . . * G

Legend: S=Start, G=Goal, *=Path, #=Obstacle, .=Empty

PS C:\Users\hamzz\OneDrive\Documents\GitHub\Semester-4>
```

Figure 1: A* Pathfinding Grid Visualization